LAB 9 · AN INTRODUCTION

The Harness

Everything you have to wrap round a working loop before you would let it run while you are asleep.



FOR	Anyone who has shipped a script that ran twice.
READ IT	Before the session. Ten minutes.

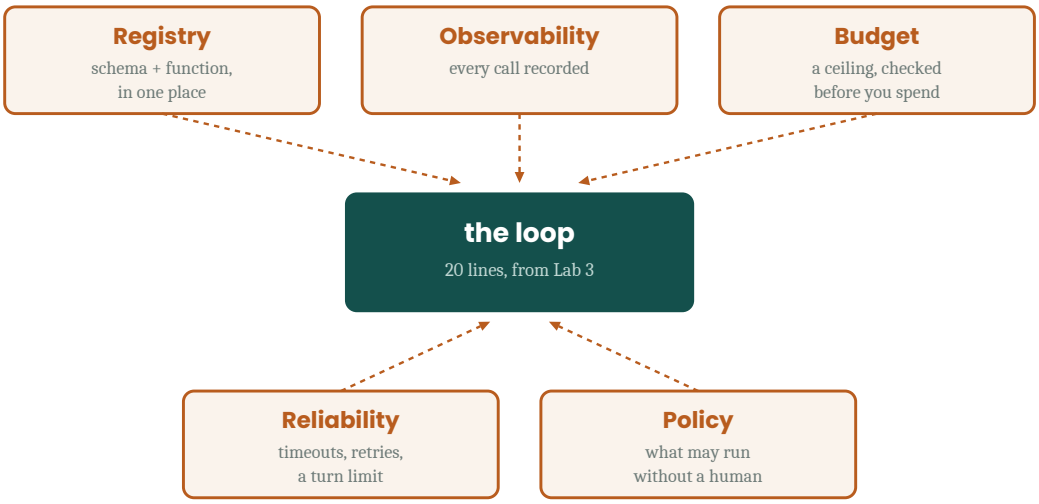
What it actually is

A harness is the code around the loop. Not the loop - the loop is twenty lines and you have already written it. The harness is the five things you bolt on before the loop can be trusted.

The agent loop from Lab 3 works. That is not in question. It worked on a Saturday afternoon, on one task, with someone watching, on a machine where nothing mattered and the inference was free.

Change any one of those conditions and it stops being enough - not because it stops working, but because when it does go wrong you have no way to find out what happened, no way to stop it, and no record of what it did.

The loop is the small part



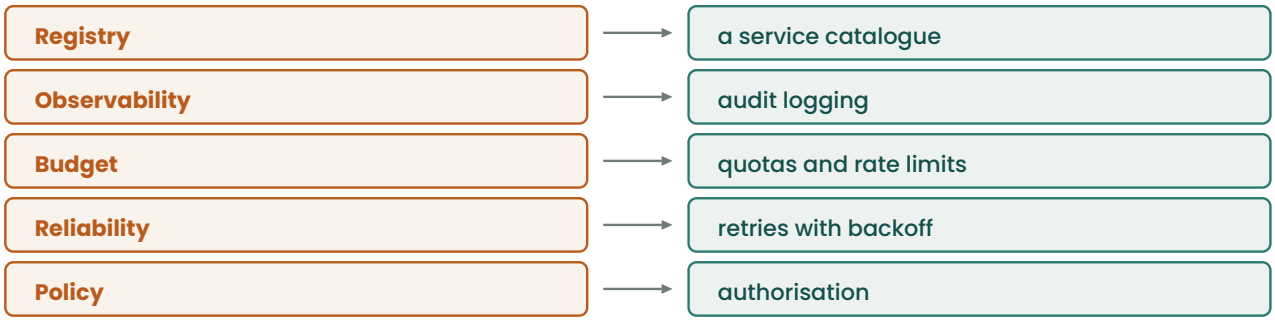
THE OBSERVATION THIS LAB IS BUILT ON

Read those five again. An audit trail. A spend cap. Retries. Authorisation. A catalogue of what can be called. **Not one of them is an AI problem.** They are the same five problems as any service that calls a remote API, and they have had the same answers for thirty years.

Which means you already know this

If you have run anything in production, you have built all five before. The vocabulary is different and the libraries are new; the problems are not.

You have solved all five before, under other names



Nothing in the right-hand column is new, and nothing in it is about AI.

This is worth saying plainly because the opposite impression is everywhere. Agentic systems are presented as a new discipline requiring new instincts. The loop is new. The engineering around it is almost entirely familiar, and the people who are good at it are the people who were already good at running services.

Example 1 - The job that runs at three in the morning

The case where every single one of the five earns its place.

An agent processes a queue overnight: for each item it reads some records, works something out, and writes a result. Nobody is watching. It handles perhaps four hundred items between midnight and six.

Run Lab 3's loop against that and here is the night you get:

00:14	Item 7 triggers a long chain of tool calls. Nothing caps it. It runs twenty turns, each resending the whole conversation, and the cost of that single item exceeds the other 399 combined.
02:40	A transient network error on one tool call raises out of the loop. The process dies. Items 150 to 400 are never attempted, and nothing says so.
08:30	You find out. The queue is backed up, the output is partial, and there is no log of what was processed, what was skipped, or what anything cost.

None of those is an exotic failure. They are Tuesday. With a harness, the same night produces: item 7 stopped at its turn limit and marked for review, the network error retried twice and succeeded, and a JSONL file with one line per event that answers every question before you have to ask it.

Example 2 - The second person to run it

The case that catches teams, because it does not look like a technical problem.

You built an agent that helps with on-call: it reads logs, checks disk, and can restart a service. You know which of those three is dangerous, so you are careful. It is useful enough that a colleague asks for it.

They do not know. Why would they? It is a tool that answers questions about servers, and nothing in it distinguishes `read_log` from `restart_service` - both are just names in a dispatch dictionary.

YOU	**The ingest service has been failing for an hour. Diagnose it and fix it.**
AGENT	Disk is full and ingest is wedged. Restarting it.
RUNS	`restart_service(name="ingest")`
OUTPUT	`Service ingest restarted. 2 in-flight requests dropped.`
AGENT	Restarted ingest. Root cause is /var at 98% full - it will fail again unless you free space.

It diagnosed correctly and then restarted a production service because the word "fix" was in the request. Nobody authorised that. The agent was not wrong, exactly - it did what it was asked - but no human was ever given the chance to say no.

> **WHY AN IF-STATEMENT IS NOT THE ANSWER**
>
> The obvious patch is `if name == "restart_service": ask()`. That works for one tool. Add a second dangerous tool and you add a second branch; add a second task with different tools and you copy the whole loop. A **policy** - a rule, held as data - can be printed, tested, reviewed in a pull request, and loaded differently in staging and production.

When you do not need one

The judgement this document exists to give you, and it points the other way more often than you would expect.

You do not need a harness for a script you run once, watching, on free inference, where the worst outcome is a wrong answer you spot immediately. Lab 3's twenty lines are genuinely the right amount of code for that. Wrapping them in five abstractions would be ceremony, and you would be maintaining infrastructure for a problem you do not have.

Write the twenty-line loop first. Every time. It is the fastest way to find out what tools the job actually needs, and you cannot design a registry for tools you have not discovered yet.

Then reach for a harness the moment any one of these becomes true:

It runs unattended.	Nobody is there to notice it looping, or to say no.
It runs more than once.	The second task is where a script becomes a pattern. A pattern with no registry is copy-paste.
Someone else runs it.	They will not know which tools are dangerous, and nothing in your code tells them.
It costs real money.	A cap you set in advance is a different object from a bill you read afterwards.
It can do something you'd regret.	Irreversible, external, or expensive. Any one of the three needs a policy, not a comment.
You have to explain it.	To a colleague, an auditor, or yourself next Tuesday. Without a trace, you cannot.

> **THE TWO WAYS PEOPLE GET THE TIMING WRONG**
>
> **Too early:** elaborate infrastructure built around a problem nobody has understood yet, abstracting tools that turned out to be the wrong tools.
>
> **Too late:** after the 3am incident, which is the expensive way to discover you needed a spend cap. The second is more common and more costly, but the first wastes more weeks.

What to take into the session

ONE

A harness is the code around the loop: a registry, observability, a budget, reliability, and a policy. The loop itself stays twenty lines.

TWO

None of the five is an AI problem. You have built all of them before, under different names, for ordinary services.

THREE

Write the loop first, every time. Reach for a harness at the second task, or the first unattended run, whichever comes first.

The session itself runs backwards from most: you spend the first half-hour watching Lab 3's loop fail four times under realistic conditions, and only then read the thing that fixes it. The test at the end is not whether the code is tidy. It is whether somebody can solve a problem you never thought about without opening your file.

The harness in the lab is about 180 lines and deliberately small enough to read in ten minutes. What it leaves out - persistence, concurrency, circuit breakers, cost attribution, a real policy engine - is the shape of a genuinely good portfolio project, and very few people have built one.